Informe de Laboratorio Nro. 2
Arquitectura del Computador

Universidad de Carabobo
Facultad de Ciencias y Tecnología

Estudiantes:
José Álvarez (31389258)
José Núñez (31581050)

20 de agosto de 2026

1. ¿Qué diferencias existen entre registros temporales ($t0-$t9) y registros guardados ($s0-$s7) y cómo se aplicó esta distinción en la práctica?

Diferencias: Los registros temporales ($t0-$t9) no garantizan la preservación de su valor a través de llamadas a funciones (son volátiles por convención), por lo que cualquier subrutina puede modificarlos libremente. En cambio, los registros guardados ($s0-$s7) deben mantener su valor inalterado tras una llamada; si una función los utiliza, tiene la obligación de respaldarlos en la pila (stack) al inicio y restaurarlos antes de retornar.

Aplicación práctica: En quicksort y mergesort, se utilizaron los registros $s0 y $s1 para conservar los límites estables del subarreglo (low/high o left/right) a través de múltiples capas de llamadas recursivas `jal`. Por el contrario, los registros $t se emplearon para cálculos inmediatos de índices, desplazamientos de memoria (`sll`) y condiciones de bucle locales que no necesitaban sobrevivir a una llamada recursiva.

2. ¿Qué diferencias existen entre los registros $a0-$a3, $v0-$v1, $ra y cómo se aplicó esta distinción en la práctica?

Diferencias:

- Los registros de argumentos ($a0-$a3) se emplean para pasar los primeros parámetros de entrada a una función.
- Los registros de valores de retorno ($v0-$v1) almacenan los resultados devueltos por la subrutina.

- El registro de dirección de retorno ($ra) almacena la dirección de memoria a la cual el procesador debe saltar (`jr $ra`) al finalizar la ejecución de la función.

Aplicación práctica: $a0 a $a3 se cargaron constantemente antes de invocar mergesort, quicksort, partition o merge para enviar la dirección base del arreglo y los límites de los subarreglos. $v0 se usó para retornar el índice del pivote en partition, y $ra se guardó rigurosamente en la pila en cada nivel recursivo para evitar perder el flujo de retorno al programa principal.

3. ¿Cómo afecta el uso de registros frente a memoria en el rendimiento de los algoritmos de ordenamiento implementados?

Impacto: El acceso a los registros del procesador es inmediato (un ciclo de reloj), mientras que las operaciones de lectura y escritura en la memoria RAM o en la pila implican latencias considerables y el uso de múltiples instrucciones (`lw`, `sw`).

Aplicación práctica: Al mantener variables de control de bucles y límites de partición directamente en los registros $s y $t, se minimizó el tráfico de memoria. Sin embargo, los algoritmos de ordenamiento recursivos exigen un uso intensivo de la pila ($sp) para almacenar estados y registros, lo que penaliza parcialmente el rendimiento en comparación con implementaciones iterativas puras.

4. ¿Qué impacto tiene el uso de estructuras de control (bucles anidados, saltos) en la eficiencia de los algoritmos en MIPS32?

Impacto: En arquitectura MIPS, la ausencia de bucles complejos de alto nivel obliga a traducir construcciones como *while* o *for* mediante combinaciones de instrucciones de salto condicional e incondicional (`bgt`, `j`). Esto incrementa el número total de instrucciones ejecutadas y puede afectar la predicción de saltos en hardware real.

Aplicación práctica: En el bucle de partición de Lomuto y en el ciclo de comparación de merge, el uso excesivo de saltos para verificar condiciones de término (`bgt, blt`) requiere un diseño cuidadoso de las etiquetas para evitar bucles infinitos o saltos en falso, afectando la cantidad de instrucciones por ciclo (IPC).

5. ¿Cuáles son las diferencias de complejidad computacional entre el algoritmo Quicksort y el algoritmo Mergesort? ¿Qué implicaciones tiene esto para la implementación en un entorno MIPS32?

Diferencias: Ambos algoritmos poseen una complejidad promedio de $O(n \log n)$. No obstante, Quicksort tiene un peor caso de $O(n^2)$ (mitigado con una buena elección de pivote), mientras que Mergesort garantiza $O(n \log n)$ en el peor caso. A cambio, Mergesort requiere memoria auxiliar adicional $O(n)$ (*temp_array*).

Implicaciones en MIPS32: Mergesort exige gestionar tanto la recursividad como la copia de datos hacia y desde un arreglo temporal estático en la sección `.data`, lo que complica la gestión de punteros y aumenta el consumo de registros temporales. Quicksort opera ïn-place", simplificando el uso de memoria, pero su implementación recursiva profunda en MIPS pone a prueba la correcta administración del *stack pointer* ($sp).

6. ¿Cuáles son las fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32 (camino de datos)? ¿En qué consisten?

Fases:

1. **Búsqueda (Instruction Fetch - IF):** Se lee la instrucción desde la memoria de instrucciones usando el contador de programa (PC).
2. **Decodificación (Instruction Decode - ID):** Se interpretan los campos de la instrucción y se leen los registros correspondientes en el banco de registros.
3. **Ejecución (Execute - EX):** La Unidad Aritmético-Lógica (ALU) realiza la operación aritmética o calcula la dirección de memoria.
4. **Acceso a Memoria (Memory Access - MEM):** Se lee o escribe en la memoria de datos (usado en instrucciones como `lw` o `sw`).
5. **Escritura (Write Back - WB):** El resultado obtenido se almacena de vuelta en el registro de destino del banco de registros.

7. ¿Qué tipo de instrucciones se usaron predominantemente en la práctica (R, I, J) y por qué?

Predominio: Se utilizaron predominantemente instrucciones de Tipo R (como `add, sub, sll, sra, move, jr`) y de Tipo I (como `addi, lw, sw, bgt, bge`). Las instrucciones de Tipo J (`j, jal`) se limitaron estrictamente a las llamadas a subrutinas y saltos incondicionales globales.

Justificación: Las instrucciones tipo R e I son indispensables en MIPS para realizar operaciones aritméticas sobre los índices del arreglo, calcular desplazamientos de memoria mediante multiplicación por 4 (`sll`) y manejar las comparaciones lógicas de los bucles de ordenamiento.

8. ¿Cómo se ve afectado el rendimiento si se abusa del uso de instrucciones de salto (j, beq, bne) en lugar de usar estructuras lineales?

Impacto: Un alto volumen de saltos interrumpe el flujo secuencial de prebúsqueda del cauce (*pipeline*) del procesador, generando riesgos de penalización por saltos (*branch*

penalties) o burbujas en el pipeline si no hay técnicas de predicción efectivas. En simuladores como MARS, aunque no modela un pipeline complejo por defecto, se traduce en un mayor número de ciclos de ejecución acumulados.

9. ¿Qué ventajas ofrece el modelo RISC de MIPS en la implementación de algoritmos básicos como los de ordenamiento?

Ventajas: Su conjunto de instrucciones reducido y uniforme permite que operaciones como sumas, desplazamientos y accesos a memoria sigan un patrón predecible de longitud fija (32 bits). Esto facilita el control explícito del flujo de datos, la manipulación directa de registros y una comprensión profunda de cómo operan a bajo nivel los algoritmos recursivos.

10. ¿Cómo se usó el modo de ejecución paso a paso (Step, Step Into) en MARS para verificar la correcta ejecución del algoritmo?

Aplicación: Permitió avanzar instrucción por instrucción (*Step*) o entrar al detalle de las funciones (*Step Into*), observando en tiempo real cómo cambiaban los valores de los registros $a0-$a3, cómo se actualizaba el puntero de la pila ($sp) al hacer llamadas recursivas, y cómo los elementos del arreglo en la memoria de datos (*data segment*) se intercambiaban o se copiaban al *temp_array*.

11. ¿Qué herramienta de MARS fue más útil para observar el contenido de los registros y detectar errores lógicos?

Herramienta: La pestaña *Registers* (para visualizar el estado de los registros temporales, guardados y el *stack pointer*) combinada con la ventana de *Data Segment* (para inspeccionar de manera visual y directa la modificación de los valores en el arreglo y el arreglo auxiliar en tiempo de ejecución). Esto fue vital para corregir desfases de índices y errores en los límites de copia.

12. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo R? (por ejemplo: add)

Visualización: Al ejecutar una instrucción de tipo R en modo paso a paso, el diagrama lógico interno de MARS (o el simulador de hardware asociado) ilumina el flujo que va desde el Banco de Registros hacia la ALU, mostrando la operación realizada y el retorno del resultado hacia el registro de destino especificado en la instrucción.

13. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo I? (por ejemplo: lw)

Visualización: Para una instrucción tipo I como `lw`, el camino de datos muestra cómo se toma la base desde un registro, se le suma el valor inmediato (*offset*) mediante la ALU, se direcciona la Memoria de Datos para extraer el contenido almacenado en esa dirección, y finalmente se dirige ese valor hacia el banco de registros para ser escrito en la posición de destino.

14. Justificar la elección del algoritmo alternativo

Justificación: Se seleccionó Mergesort como contraparte de Quicksort porque ambos representan el paradigma de "Divide y Vencerásçon la misma complejidad algorítmica promedio $O(n \log n)$, pero con enfoques contrastantes de ingeniería: Quicksort destaca por su eficiencia en espacio (*in-place*) con particionamiento de Lomuto, mientras que Mergesort evidencia la necesidad de gestionar memoria auxiliar (*temp_array*) y un proceso estricto de combinación (*merge*), lo que enriquece el análisis comparativo de la gestión de memoria y registros en MIPS32.

15. Análisis y Discusión de los Resultados

Discusión: La implementación exitosa de ambos algoritmos en el entorno MARS demostró que la complejidad teórica de los algoritmos de ordenamiento se traduce directamente al nivel de ensamblador en un aumento en la complejidad de control de registros y manejo de pilas. Mientras que Quicksort requirió un control minucioso de los índices de partición y llamadas recursivas simples, Mergesort exigió un manejo avanzado de búferes temporales y múltiples punteros simultáneos (i, j, k). Los resultados confirmaron que un diseño metódico del prólogo y epílogo de las funciones previene la corrupción de datos y garantiza la estabilidad de las estructuras recursivas en arquitecturas de tipo RISC.